

访谈纪要：Tony Pan — Vercel

2026-04-07 20:41

目录

访谈纪要：Tony Pan — Vercel

一、当前角色与公司环境

二、工具矩阵：三种工具的精准分工

三、开发工作流：端到端的 AI 驱动流程

阶段一：需求理解 → 与 AI Brainstorm

阶段二：AI 创建 Linear Tickets + 设计文档

阶段三：并行开发（Git Worktree 多实例轮转）

阶段四：30 Agent 并行 Code Review

四、工作节奏的变化：从深度专注到高频切换

六、AI 时代的痛点与焦虑

痛点一：AI 质量的不一致性

痛点二：并行过多导致人的认知过载

痛点三：AI 产出速度超过人的处理能力——人反而成了瓶颈

七、技术选型：开发者如何判断一个项目/工具是否靠谱

七、Vercel 的开发者营销（Developer Experience 内部视角）

九、信息获取渠道

九、附录：四家科技公司 AI 工具政策对比（内部视角）

关键金句

用户画像表格

与其他受访者的对比

访谈纪要：Tony Pan — Vercel

项目	信息
受访者	Tony Pan (Vercel Software Engineer)
人群分类	A2 — 大型企业资深工程师 / AI 重度用户。15年+经验，横跨 Google、Meta、xAI、Vercel 四家顶级科技公司，已完全不手写代码，用 AI 完成所有开发工作
访谈日期	2026-04-07
访谈人	陈诗园 (shiyuan)
录制链接	录制文件： https://meeting.tencent.com/crm/24DwBgog3d
访谈时长	约 55 分钟
访谈语言	英文
受访者背景	15-16年软件工程经验。工作经历：Meta (2015-2020, 约5年) → Google (2020-2024, 约4年) → Meta (2024-2025, 约半年) → xAI (2025.2-2025.3, 约1个月) → Vercel (2025.4至今)。Vercel 是约 700 人的后期创业公司，Tony 任资深工程师 (fairly senior engineer)，负责构建高性能可扩展系统。每天使用 AI 工具，6-12个月经验。使用 Claude Code (主力)、Codex、Cursor (轻量任务)，公司提供无限 token，个人不额外付费。

一、当前角色与公司环境

Tony 目前在 Vercel 担任资深工程师，负责构建高性能可扩展的内部系统 (proprietary systems)。Vercel 是一家约 700 人的后期创业公司，以 Next.js 和前端部署平台闻名，客户群体主要是开发者。

Vercel 对 AI 的态度在受访公司中最为激进——公司为所有员工提供无限 AI token (Claude Code、Codex、Cursor 全部免费)，并且内部有一个 **token 使用量排行榜**，鼓励员工尽可能多地使用 AI。Tony 说自己甚至排不到前面，说明公司内部存在大量比他更激进的 AI 使用者。这种“无上限鼓励”的环境，催生了很多在其他公司不可能出现的创新用法 (如 30 agent 并行 code review)。

"My company has unlimited tokens. They actually look at a leaderboard of who uses the most tokens. And I'm not even on top of the list. People do lots of crazy things with it."

「我们公司有无限制 token。公司有一个 token 使用量排行榜，我都排不到前面，有人用得比我还疯狂。」

Tony 已经**完全不手写代码**，所有开发工作通过 AI 完成。他甚至用 Claude Code 来读 Slack 消息、写 Notion 文档、做项目规划，AI 已经成为他与所有工作工具之间的统一接口。

"I haven't written any line of code for a long time now. Completely with using AI assistance."

「我已经很久没写过一行代码了。完全用 AI 完成。」

二、工具矩阵：三种工具的精准分工

Tony 同时使用三种 AI coding 工具，但不是随意切换，而是形成了清晰的**分层架构**：Claude Code 负责所有核心开发，Codex 作为独立验证层确保质量，Cursor 只处理 Slack 上报来的轻量修复。这种分工背后的逻辑是——**不信任任何单一模型的输出，用多模型交叉验证来对冲 AI 的随机性**。

工具	定位	使用场景	优劣
Claude Code	主力开发工具	所有核心开发：理解需求、项目规划、写代码、code review、写文档（Notion MCP）	能力最强，可深度定制（Skills）；但近期质量下降明显
Codex（Open AI）	质量验证层	Claude Code 输出不确定时做 double check；code review 时作为独立第二意见	近期质量追上 Claude；不直接用其 CLI
Cursor	轻量快速修复	通过 Slack 集成：有人报 bug → @Cursor fix this → 自动生成 PR	Slack 集成好用，极低摩擦；代码质量不如本地 Claude Code

Tony 不使用 Cursor 的 IDE 界面——他有大量 VS Code 个性化配置，Cursor 导入时会搞乱这些设置。他只通过 Slack bot 的方式使用 Cursor，本质上把 Cursor 当作一个“Slack 里的自动修 bug 机器人”。

Claude Code 近期质量下降是 Tony 反复强调的问题，这与 Hacker News 社区的普遍反馈一致。他怀疑 Anthropic 为了省钱降低了模型能力，导致 one-shot（一次性完成任务）的成功率明显下降。与此同时，Codex 的质量反而追上来了，两者的差距正在缩小。

"For Claude Code recently, the quality has deteriorated a lot. I think they just dumbled down their model to save money or something. The recent sentiment is Claude Code quality has been declining, and Codex actually caught up."

「Claude Code 最近质量下降了很多。我觉得他们可能为了省钱降低了模型能力。最近的舆论是 Claude Code 质量下降，Codex 反而追上来了。」

三、开发工作流：端到端的 AI 驱动流程

Tony 的开发流程是迄今受访者中**最系统化的全 AI 化流程**——从需求理解到项目规划到并行执行，每个阶段都深度依赖 AI，人的角色从“写代码的执行人”转变为“与 AI 协作的项目管理者”。

阶段一：需求理解 → 与 AI Brainstorm

拿到需求和 spec 后，**第一件事不是动手写代码，而是喂给 Claude Code 一起 brainstorm**。反复讨论直到 Tony 对需求和实现路径都足够清晰，然后让 AI 拆分成带依赖关系的里程碑。这个阶段 Tony 强调“花很长时间”——不急于写代码，而是确保方向正确。

"First thing I do is I feed it into Claude Code — explain to me what this is, and how we can approach implementing it. We start brainstorming until I have a fairly good understanding. That takes quite a while."

「第一件事是把需求喂给 Claude Code——解释这是什么、怎么实现。我们一起 brainstorm 直到我充分理解。这会花不少时间。」

阶段二：AI 创建 Linear Tickets + 设计文档

让 Claude Code 自动在 Linear（项目管理工具）中创建 tickets，设定依赖关系和 critical path。Tony 对 ticket 的要求很高——**每个 ticket 必须包含足够的上下文，让任何工程师或 AI agent 都能独立执行**。AI 在理解依赖关系和创建结构化任务方面表现特别好。同时让 AI 撰写设计文档，经团队 review 后开始执行。

"I put in enough context so that anyone who picks it up — either engineer or a bot — can have enough context to work on it independently."

「每个 ticket 放足够的 context，无论是工程师还是 bot 拿到都能独立完成。」

阶段三：并行开发（Git Worktree 多实例轮转）

因为 AI 响应需要时间，Tony 用 **Git worktree** 为每个任务创建独立工作目录，每个上面跑一个 Claude Code 实例，日常同时推进 **3-5 个任务**——给 A 下指令 → 切到 B 看结果 → 给 B 下指令 → 切到 C，循环往复。这种“轮转式并行”是他利用 AI 等待时间的核心策略。

"I have a Claude Code running on each work tree. I would rotate between them — do something here, tell it to do something next, switch to the next tab."
「每个 worktree 上跑一个 Claude Code。在它们之间轮转——这边下指令，切到下一个看结果，再下指令。」

阶段四：30 Agent 并行 Code Review

因为 AI 的 code review 具有很大的随机性——同一段代码审查 5 次会得到 5 组不同的发现——Tony 的解法是以**量取胜**。他写了一个 Skill，每次 code review 自动启动 30 个 AI agent 并行审查，分为三组，每组使用不同的视角和模型：

"AI is kind of random. If you ask to review the code 5 times, it will come up with 5 different things."
「AI 有很大的随机性。你让它审查同一段代码 5 次，会得到 5 组不同的发现。」

Agent 组	数量	模型	Prompt 策略
组 1：标准审查	10个	Claude Code	"Review this PR"
组 2：独立第二意见	10个	Codex	"Review this PR"（不同模型提供多样性，避免 Claude 的系统性盲区）
组 3：事故模拟	10个	Claude Code	"This PR caused a big incident, root cause it"

第三组的 prompt 通过改变角色框架——从“帮我看看这代码”变成“这代码出了重大事故”——让 AI 以事故调查员的心态审查代码，从而关注并发问题、边界条件、资源泄漏等在标准 review 中容易被忽略的维度。

"The last 10 is Claude Code agents, but I don't tell them to review — I say 'this PR caused a big incident, and root cause it.' That's how I trick the AI into being more thorough."
「最后 10 个 Claude Code 我不说 review——我说‘这个 PR 导致了重大事故，做 root cause’。这样骗 AI 让它更仔细。」

30 个 agent 完成审查后（约 45 分钟，期间会被 rate limit 限制需要重试），由一个主 Claude Code agent 汇总所有发现，按**严重程度**和**发现频次**（有多少个 agent 独立发现了同一个问题）排序。Tony 再逐条人工判断每个发现是真实问题还是幻觉。他有时会故意在代码中保留一个已知的问题不告诉 AI，观察 30 个 agent 能否发现——如果发现了，说明这轮 review 质量够高；如果没发现，就再跑一轮。最终让 AI 自动将确认的评审意见发布到 PR 上。

"The main Claude Code agent summarizes all findings, ranks them based on how severe things are and how many agents found it. The quality is very high, because the randomness evens out."
「主 Claude Code agent 汇总所有发现，按严重程度和发现频次排序。质量非常高，因为随机性被平均掉了。」

"Sometimes I find something wrong with the PR, but I'm not gonna tell the AI. I let it review. If it finds it, the review is good. If it doesn't, I do another round."
「有时我发现 PR 里有问题但不告诉 AI。如果它找到了，说明 review 质量好。如果没找到，再来一轮。」

每次 review 消耗约 **\$50 token**，需要 45 分钟运行时间。这在个人项目中不可行，但在 Vercel 无限 token 的环境中成为可能。整个流程已封装为 Skill，一个命令启动。

"You can easily burn like \$50 of tokens per review per PR. But for a company, they encourage it."
「一次 PR review 很容易烧掉 \$50 token。但公司鼓励这么做。」

四、工作节奏的变化：从深度专注到高频切换

AI 时代对工程师角色的影响，Tony 给出了迄今受访者中最深刻的洞察：**代码能力已经不再是区分工程师优劣的因素**，因为 AI 让每个人都能写出差不多水平的代码。真正区分工程师的，变成了过去属于 PM 的能力——发现问题、提出方案、推动跨团队协作、向上管理。这意味着工程师的绩效考核标准正在从“实现能力”转向“策略和影响力”。

"The coding ability is no longer a distinguishing factor for engineers. What distinguishes you from your peers is your ability to come up with ideas, to drive the project direction — things like soft skills, PM skills. Cause every one can code now. There's no bad software engineers anymore."

「代码能力不再是区分因素。区分你的是提出想法的能力、驱动项目方向的能力——更像是 PM 技能。因为人人都能写代码了，没有差的工程师了。」

Tony 坦言这对他个人也是挑战。作为一个 15 年经验的工程师，他习惯了被给一个困难的技术问题然后深入钻研。但现在“一个 prompt 就搞定了”，他不得不将精力转向观察用户行为、分析数据、和 PM 讨论方向——这些过去不是他的核心工作。

"I'm used to just being given a very difficult technical problem and diving into it. But now, it's just one prompt and everything's figured out."

「我习惯了被给一个很难的技术问题然后钻进去。但现在一个 prompt 就搞定了。」

PM 也在变化——Vercel 的 PM 已经在用 Cursor 直接做小修改、提 PR，只是还需要工程师 review。Tony 认为 PM 直接写代码的趋势会继续深化，工程师和 PM 的边界正在模糊。

六、AI 时代的痛点与焦虑

痛点一：AI 质量的不一致性

Tony 最头痛的是 **Claude Code 近期质量波动大**，有时候很聪明，有时候很“笨”，one-shot 能力明显退步。这迫使他投入大量时间做人工审查，甚至要多次迭代才能得到可用的输出。30 agent review 系统本身就是为了应对这个问题而生。

"The biggest pain point is how to make sure the AI's quality is consistent. It's very frustrating sometimes, trying to guide it toward the right way."

「最大痛点是如何确保 AI 质量一致。引导它走正确方向有时非常令人沮丧。」

痛点二：并行过多导致人的认知过载

为了利用 AI 等待时间，Tony 同时推进多个任务（Git worktree 并行），但经常失控——开了 10 个 tab 后完全记不清每个 tab 在做什么，甚至要问 AI “我之前在做什么”。传统工程师被训练深度专注一个问题，AI 时代却要求高频上下文切换，这种转变让人**更累而非更轻松**。

"I find myself with 10 different tabs, completely lose track of what I was trying to do. I go to a new tab and ask the AI — 'what was I doing?'"

「10 个 tab，完全记不清自己在干什么。切到新 tab 问 AI——‘我之前在做什么？’」

"That makes you a lot more tired. You feel like you've been in meetings all day with 100 people."

「这让你更累。感觉像跟 100 个人开了一天不同的会。」

痛点三：AI 产出速度超过人的处理能力——人反而成了瓶颈

Tony 总结的最大痛点不是某个具体的工具问题，而是**人的精力和认知能力跟不上 AI 的产出速度**。AI 能做的越来越多、越来越快，但人需要审查、判断、决策，这些环节反而成了拖慢整体效率的瓶颈。他用了一个看似反直觉的说法——“humans are not the bottleneck”——实际表达的意思是：现在的瓶颈不在 AI 那边，而在人这边。

"The biggest pain point is — humans are not the bottleneck. I look forward to a future where there's technology that takes us out of the loop, or makes it easier for us to work better with AI."

「最大的痛点是——瓶颈不在 AI，而在人。我期待未来有技术能把我们从循环中解放出来，让我们更好地与 AI 协作。」

这与其他受访者的痛点形成对比：Daniel 觉得 AI 太贵，Ervin 觉得选择太多，Binita 觉得推广难——Tony 的痛点已经超越工具层面，指向人在 AI 时代的角色问题。

七、技术选型：开发者如何判断一个项目/工具是否靠谱

与之前的创业者受访者（Daniel 让 Claude Code sub-agent 做 research）不同，Tony 的技术选型方式很传统——不用 AI 做评估，而是依赖经验和人工检查。他看的指标是 repo 本身的健康度，而非 AI 的推荐。

评估维度	具体做法
NPM 下载量	去 NPM 看周下载量——500 下载 = 不靠谱，百万级 = 可信任
GitHub Stars	看 stars 数量，作为社区认可度的快速指标
活跃度	看最近一次 commit 时间——应该是最近几天内；项目活跃意味着有人在维护，出问题能修
贡献者背景	查主要贡献者的 LinkedIn / GitHub 背景，看他们是否有相关领域经验，是否"legit"（靠谱）
包体积	看依赖包大小——避免为小功能引入巨大依赖（前端尤其敏感，影响加载速度）
内部替代	优先选公司已经在用的工具，即使不是 100% 完美——已被审查过的工具更安全

"I would go to NPM and look at downloads. Go to GitHub and look at stars. Look at whether a project is being actively maintained. Look up the major contributors' LinkedIn or GitHub profiles to see if they are legit."

「去 NPM 看下载量，GitHub 看 stars，看项目是否活跃，查贡献者的 LinkedIn 看是否靠谱。」

Vercel 有 IT 维护的合规工具清单，新工具需过安全审查。OpenClaw（安全风险）已从公司电脑封禁，其余通过企业协议使用。Notion MCP 曾因安全审查被 IT 短暂禁用，但因内部需求太大，IT 最终"走回头路"允许使用——这说明开发者的使用需求在某些情况下可以倒逼安全策略调整。

七、Vercel 的开发者营销（Developer Experience 内部视角）

作为开发者工具公司的内部工程师，Tony 提供了一个从内部看 Vercel 如何做 developer experience（简称DX）的视角。

核心产品魔法：Git Push → 自动部署。 Tony 认为 Vercel 最成功的一点是**极大降低了"改代码"到"看到效果"之间的摩擦**——把 Git 仓库和网站绑定，每次推代码就自动构建部署，不喜欢就像 Git 回滚一样回退。所有其他开发者体验都建立在这个简单而强大的基础之上。

"There's something really magical about it — you push code, and the website changes. It drastically reduces the friction."

「有种魔法感——你推代码，网站就变了。大幅减少了摩擦。」

DX 设计哲学："一条命令搞定"。 作为开发者给开发者做产品，Vercel 团队天然理解什么样的体验是"好"的——完成一件事应该只要一条命令，而不是一个巨大的配置文件。命令行工具、Dashboard、GitHub 集成都围绕这个原则设计。

"If you want to get something done, it should be just one command, instead of a giant config file."

「做一件事应该只要一条命令，而不是一个巨大的配置文件。」

高管直接与开发者互动。Vercel 的创始人、高管和 Developer Experience 工程师都活跃在 Twitter 上，实时追踪客户群体的情绪。CEO 会直接回复抱怨的客户、把推文转给工程团队修复——这种“高管-客户-工程”的短路径反馈循环，让 Vercel 能快速响应开发者社区的声音。

"Our CEO would directly engage with customers complaining, forward us the tweet, and then we fix something."

「CEO 会直接回复抱怨的客户，把推文转给我们，我们就修。」

内部 Hackathon = 向上展示的机会。Vercel 定期举办全公司 Hackathon，因为公司只有 700 人、领导层都是技术出身，所以 Hackathon 实际上是工程师在管理层面展示能力和影响力的重要舞台。几乎默认每个人都要参加。

九、信息获取渠道

Tony 的信息获取渠道相比之前的创业者受访者（Sammy 用 OpenClaw 24/7 监控、Daniel 用 X/Twitter）更为传统——主要依赖 Hacker News 首页和 YouTube。他自己不用 Twitter，通过公司 Slack 频道间接获取 Twitter 上的信息。

渠道	用途	频率
Hacker News（首页）	核心信息源——新模型、新工具、新趋势	每天
Hacker News Weekly Newsletter	每周摘要	每周
内部 Slack 频道	同事分享 AI 新闻（替代 Twitter）	每天
YouTube — Theo	最新 AI 八卦和动态，娱乐性强	每天
YouTube — Fireship	AI 工具/技术失败案例总结	定期
YouTube — AI King	AI 新模型和工具的快速总结	定期
ByteByteGo Newsletter	系统设计内容，只看标题	每周
模型提供商内部渠道	Vercel 与模型提供商合作，提前知道新模型	不定期

学习新技术的方式也值得注意——Tony 会找到新技术的 GitHub repo，然后让 Claude Code 读源代码来学习，而不是只给名字让 AI 解释。他还写了一个 Teach Skill，让 AI 把复杂主题拆解成渐进式模块逐步教学，以及一个 Interview Skill 让 AI 用难题拷问自己，迫使他真正理解概念。

Vercel 内部有一个共享 Skill 仓库（内部 repo），员工可以把好用的 Skill 提交上去供所有人 clone 使用。Tony 也有一套不分享的私人 Skill 集合。

Tony 不焦虑——他认为身处 Vercel 已经在前沿了（公司与模型提供商合作，提前知道新模型信息）。对于每天涌现的新框架和工具，他的态度是等它成熟再说。

"It's silly to chase the latest trend. Most of them won't turn into anything. It's not worth the time to investigate all of them."

「追最新潮流挺傻的。大多数不会成气候，不值得逐一研究。」

九、附录：四家科技公司 AI 工具政策对比（内部视角）

Tony 在 Google、Meta、xAI、Vercel 都工作过，这使他能从内部视角对比不同公司的 AI 工具策略。最讽刺的发现是：最有能力提供 AI 工具的大厂（Google、Meta），反而是限制最严的，而 Vercel 这样的后期创业公司提供了最开放的环境。

公司	时间	AI 工具政策	Tony 的体验
Meta (第二次)	2024.8-2025.2	强制推广 AI，但只允许用 Meta 内部模型，禁止 Claude Code 等外部工具	内部模型太差，Tony 一度以为 AI coding 没有前途
Google	2020-2024	只允许 Gemini 模型；DeepMind 团队例外可用 Claude（讽刺）	听到过同事对只能用 Gemini 的抱怨
xAI	2025.2-2025.3	未展开	只待了一个月
Vercel	2025.4至今	为所有员工提供无限 token（Claude Code + Codex + Cursor），鼓励使用，有 token 使用量排行榜	Tony 的 30 agent review 等创新用法成为可能

"When I was at Meta, everybody had to use Meta's internal, crappy model. It was so bad that I thought AI was not gonna take off."

「在 Meta 必须用内部那个很差的模型。差到我以为 AI 根本不会成功。」

"It was not until I came out of Meta, and I tried other models, that I realized — this is actually gonna make a difference."

「直到离开 Meta 试了其他模型，才意识到——这玩意儿真能改变一切。」

关键金句

主题	金句（英）	金句（中）
完全不写代码了	"I haven't written any line of code for a long time now."	「我已经很久没写过一行代码了。」
人反而成了瓶颈	"The biggest pain point is — humans are not the bottleneck."	「最大的痛点是——瓶颈不在 AI，在人。」
代码能力不再区分工程师	"There's no bad software engineers anymore."	「没有差的软件工程师了。」
欺骗 AI 做更好的 Review	"I say 'this PR caused a big incident, root cause it.' That's how I trick the AI into being more thorough."	「我说'这个 PR 导致了重大事故，做 root cause'——这样骗 AI 让它更仔细。」
上下文切换的疲惫	"You feel like you've been in meetings all day with 100 people."	「感觉像跟 100 个人开了一天不同的会。」
失控的并行	"I go to a new tab and ask the AI — 'what was I doing?'"	「切到新 tab 问 AI——'我之前在做什么？'」
Meta 内部模型太差	"It was so bad that I thought AI was not gonna take off."	「差到我以为 AI 根本不会成功。」
离开大厂才知道	"It was not until I came out of Meta that I realized — this is actually gonna make a difference."	「直到离开 Meta 才意识到——这玩意儿真能改变一切。」
Claude 质量下降	"I think they just dumbed down their model to save money."	「我觉得他们为了省钱降低了模型能力。」
Vercel 的魔法	"You push code, and the website changes. There's something really magical about it."	「你推代码，网站就变了。有种魔法感。」
DX 哲学	"It should be just one command, instead of a giant	「应该只要一条命令，而不是一个巨大的

主题	金句（英）	金句（中）
	config file."	配置文件。」
Token 排行榜	"They have a leaderboard of who uses the most tokens. I'm not even on top."	「公司有 token 排行榜，我都排不到前面。」

用户画像表格

维度	详细信息
用户名称	Tony Pan
人群分类	A2 — 大型企业资深工程师 / AI 重度用户。15年+经验，横跨 Google/Meta/xAI/Vercel，已完全不手写代码
代码背景	15-16年软件工程经验，资深工程师级别。曾在 Meta（5年+）、Google（4年）、xAI（1个月）工作。擅长构建高性能、可扩展系统
当前角色	Vercel Software Engineer（fairly senior），公司约 700 人。负责构建和维护高性能可扩展系统
核心项目	公司内部 proprietary systems（未披露具体细节）
使用工具	主力 ：Claude Code（所有核心开发工作）； 验证 ：Codex（double check + 独立 code review）； 轻量 ：Cursor（仅 Slack 集成，自动生成 PR）； 效率 ：Notion MCP（通过 Claude Code 读写文档）； Skills ：大量自定义 Skill（code review、教学、面试练习等）；公司提供全部工具 + 无限 token
月度工具支出	\$0 个人支出（公司提供无限 token + 全部工具订阅）
工具变迁	Meta 内部模型（极差）→ 离开大厂后发现外部工具的真正实力 → 现在 Claude Code 为核心 + Codex 为验证层 + Cursor 为轻量层
决策路径	传统人工型 。SDK/依赖选型不用 AI，依赖经验和人工检查：NPM 下载量 + GitHub stars + 最近 commit + 贡献者 LinkedIn 背景 + 包体积 + 内部替代
信息触媒	核心 ：Hacker News 首页（每天）+ Hacker News 周报； 内部 ：Slack AI 新闻频道（替代 Twitter）； 视频 ：YouTube（Theo、Fireship、AI King）； Newsletter ：ByteByteGo（只看标题）； 内部渠道 ：与模型提供商合作提前知道新模型
筛选去噪	不追最新趋势，等技术成熟后再关注。不焦虑——公司本身在前沿，有内部信息渠道
核心方法论	① 30 agent 并行 code review（10 Claude + 10 Codex + 10 "事故模拟" Claude）；② Git worktree 多实例并行开发；③ 所有重复工作封装为 Skill；④ AI 做项目规划和 ticket 创建；⑤ 先与 AI brainstorm 再执行——不急于写代码

与其他受访者的对比

维度	Tony (Vercel)	Daniel (Photon)	Binita (Yelp/前Google)	Samir (Polarity)	Ervin (CxTransform)
角色定位	大厂资深工程师 (15年+经验)	种子轮创业公司联合创始人	大厂 PM + AI 先锋	创业团队 CM O/增长 + 技术	中小企业技术服务商
公司规模	~700 人 (后期创业公司)	不到 10 人	Yelp 大型企业	4 人	约 6 人
AI 工具核心	Claude Code + Codex + Cursor (三工具精准分工)	Claude Code (从 Cursor 全面迁移)	Cursor + Claude/GPT/Gemini 多模型切换	Claude Code + OpenClaw	Claude Code + Bubble
月度 AI 支出	\$0 个人 (公司无限 token)	\$10,000–20,000 (团队)	\$20 个人 + 公司承担其余	~\$1,050	~\$200
开发方法论	AI brainstorm → tickets → 并行开发 → 30 agent review	Spec-driven, 人写框架 + AI 迭代	Personal OS + sub-agent 并行	Spec-driven + Voice-first	AI 整理需求 → Claude Code 从零构建
技术选型方式	传统人工: NP M + GitHub + 贡献者背景	AI 驱动: sub-agent research → edge case challenge	问题驱动 → AI 探索 → tech sourcing team	Firecrawl + Tavily	AI 自动推荐
核心痛点	人跟不上 AI 产出节奏 + AI 质量不一致	价格太贵 + AI 缺乏自我迭代	推广难 + 量化困难 + 安全合规	不焦虑	选择太多
核心洞察	代码能力不再区分工程师, PM 技能才是	屏幕代码占比 → 心智转变	企业 AI adoption 一手视角	OpenClaw 深度自动化	非技术创始人用 AI 降低门槛
信息获取	Hacker News + 内部 Slack + YouTube	X/Twitter + LinkedIn	公司博客 + GitHub + Newsletter + Instagram	X/Twitter (24/7 agent 监控)	Instagram + YouTube + 口碑